

Main — View model — Repo — Dao / Network

Android Network 기초 (HTTP Communication)



목차

Android Network 개요

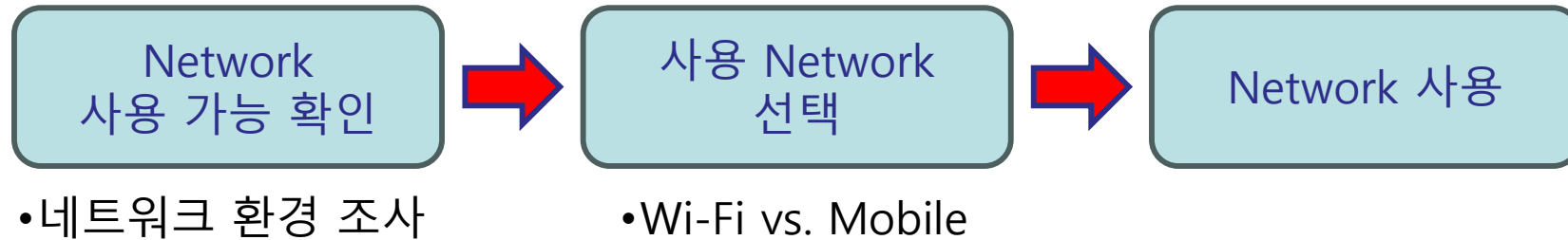
- ◆ ConnectivityManager

HTTP 통신

- ◆ HTTP 요청
- ◆ HTTP 데이터 서버 전송

네트워크 사용 개요

네트워크 사용 절차



네트워크 환경 조사

◆ConnectivityManager 사용 → NetworkInfo 확인

◆필요 퍼미션 :

→ `android.permission.ACCESS_NETWORK_STATE`

◆주요 함수 결과 (getter사용값)

- `allNetworkInfo` : 기기가 제공하는 모든 Network 타입 조사
- `activeNetworkInfo` : 활성화 상태의 Network 타입 조사
- `getNetworkInfo(type: Int)` : 특정 Network 타입 조사

Android
Manifest

네트워크 사용 확인 1

네트워크 환경 조사의 예

출처: <https://developer.android.com/training/basics/network-ops/managing.html>

```
private fun getNetworkInfo() {
    val connMgr = getSystemService(Context.CONNECTIVITY_SERVICE) as ConnectivityManager
    var isWifiConn: Boolean = false
    var isMobileConn: Boolean = false

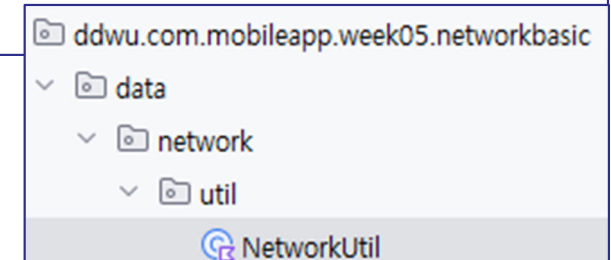
    connMgr.allNetworks.forEach { network ->
        connMgr.getNetworkInfo(network)?.apply {
            if (type == ConnectivityManager.TYPE_WIFI) {
                isWifiConn = isWifiConn or isConnected
            }
            if (type == ConnectivityManager.TYPE_MOBILE) {
                isMobileConn = isMobileConn or isConnected
            }
        }
    }

    Log.d(TAG, "Wifi connected: $isWifiConn")
    Log.d(TAG, "Mobile connected: $isMobileConn")
}
```

allNetworks →
Network 배열 반환

◆ NetworkInfo 클래스

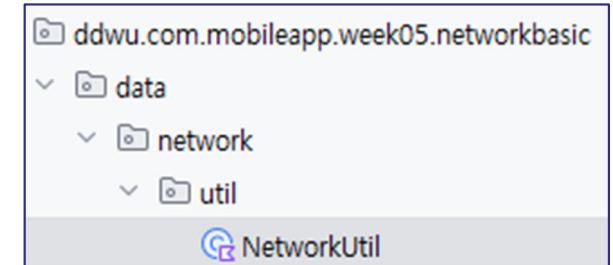
- 네트워크 타입 정보를 표현하는 클래스
- 현재 사용가능한 네트워크 확인



네트워크 사용 확인 2

네트워크 사용 전에 사용가능 확인 필요

네트워크 사용 가능 확인 예



```
private fun isOnline(): Boolean {
    val connMgr = getSystemService(Context.CONNECTIVITY_SERVICE) as ConnectivityManager
    val networkInfo: NetworkInfo? = connMgr.activeNetworkInfo
    return networkInfo?.isConnected == true
}
```

false면 알람으로 네트워크 스위치 켜라고 알려주면 됨 출처: <https://developer.android.com/training/basics/network-ops/managing.html>

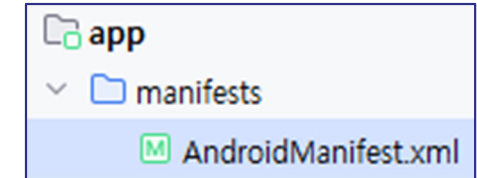
◆ ConnectivityManager.*activeNetworkInfo*

- 현재 활성화 상태의 네트워크 정보를 NetworkInfo 객체로 반환

사용 Permission

- ◆ AndroidManifest.xml 에 추가
- ◆ `android.permission.ACCESS_NETWORK_STATE`

참고 - http 주소 접속 시 고려사항



- 안드로이드 9.0 이상부터 https 만을 적용
 - ◆ http://... 주소에 접속할 경우 예외사항 발생

처리방법 1 : 서버가 https 지원 시 https:// 로 변경

처리방법 2 : 보안관련 설정 추가

- 방법1: AndroidManifest 의 <application> 속성에 속성값 추가
 - android:usesCleartextTraffic= " true "
- 방법2: res/xml/network...xml 파일 추가 후 AndroidManifest 에 지정

network_security_config.xml

```
<?xml version="1.0" encoding="utf-8"?>
<network-security-config>
  <base-config cleartextTrafficPermitted="true" />
</network-security-config>

<!--
특정 도메인(예: cooling.dongduk.ac.kr) 만 허용할 때
<?xml version="1.0" encoding="utf-8"?>
<network-security-config>
  <domain-config cleartextTrafficPermitted="true">
    <domain includeSubdomains="true">cooling.dongduk.ac.kr</domain>
  </domain-config>
</network-security-config>
```

```
<application
  android:allowBackup="true"
  android:icon="@mipmap/ic_launcher"
  android:label="MultipleAsyncTaskTest"
  android:roundIcon="@mipmap/ic_launcher_round"
  android:supportsRtl="true"
  android:theme="@style/AppTheme"
  android:networkSecurityConfig="@xml/network_security_config">
  <!-- 안드로이드 9.0 파이 버전 이상일 때 지정 필요
  또는 android:usesCleartextTraffic="true" 사용 -->
```

네트워크 사용 - HTTP 통신 1

HTTP 기반의 네트워크 활용 기본

- ◆ JAVA 기반의 네트워크 관련 API 이용
- ◆ 필요 permission (AndroidManifest.xml)
 - android.permission.INTERNET
 - android.permission.ACCESS_NETWORK_STATE

HTTP Connection 설정

- ◆ URL 클래스를 사용하여 접속할 서버 url 설정
- ◆ URLConnection (추상 클래스)를 사용하여 서버와의 연결 확보
 - HttpURLConnection 또는 HttpsURLConnection (구현 클래스)

```
val strUrl = "https://cs.dongduk.ac.kr" // https 일 경우 HttpsURLConnection 사용
val url : URL = URL(strUrl)
val conn : HttpURLConnection
    = url.openConnection() as HttpURLConnection
```

URLConnection 을 사용하여 네트워크 연결

requestMethod: "GET" 또는 "POST"

```
@Throws(IOException::class, SocketTimeoutException::class)
private fun getConnection(requestMethod: String, address: String, data: String?) : HttpURLConnection? {
    val url = URL(address)
    var conn = url.openConnection() as HttpURLConnection

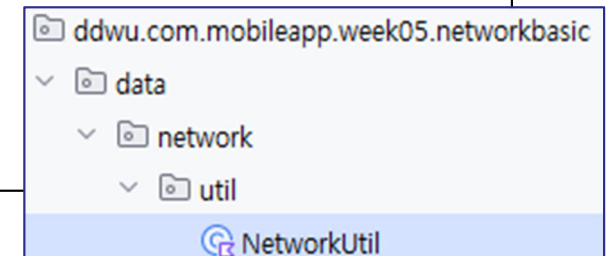
    conn.readTimeout = 5000 // 읽기 타임아웃 지정 - SocketTimeoutException
    conn.connectTimeout = 5000 // 연결 타임아웃 지정 - SocketTimeoutException
    conn.doInput = true // 서버 응답 지정 - default
    conn.requestMethod = requestMethod // 연결 방식 지정 - GET or POST

    if (requestMethod.equals("POST")) {
        conn.doOutput = true
        conn.setRequestProperty("content-type", "application/x-www-form-urlencoded; charset=UTF-8")
        val params = "subject=" + data // 여러 데이터를 추가할 경우 key=value 형태로 추가
        val outputStreamWriter : OutputStreamWriter = OutputStreamWriter(conn.outputStream, "UTF-8")
        val writer : BufferedWriter = BufferedWriter(outputStreamWriter)
        writer.write(params) // params 의 개수만큼 write
        writer.flush()
    }

    // conn.connect() // 통신 링크 열기 - 트래픽 발생
    val responseCode = conn.responseCode // 서버 전송 및 응답 결과 수신
    if (responseCode != HttpURLConnection.HTTP_OK) {
        throw IOException("Http Error Code: $responseCode")
    }
    return null
}

return conn
}
```

requestMethod가 POST
일 경우 추가 처리



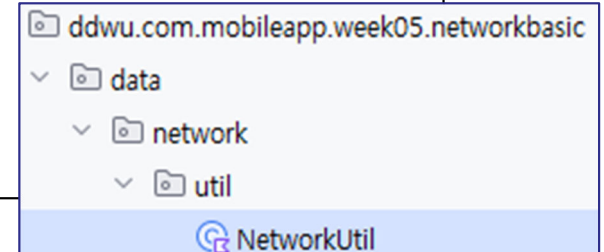
네트워크 데이터 전송 및 수신 2 – GET 요청 동덕여자대학교

GET 방식으로 텍스트 수신

◆ Web Page, XML, JSON 등

```
fun downloadText(address: String) : String? {  
    var receivedContents : String? = null  
    var resultStream : InputStream? = null  
    var conn : HttpURLConnection? = null  
  
    try {  
        conn = getConnection("GET", address, null)  
        resultStream = conn?.inputStream // 응답 결과 스트림 확인  
        receivedContents = streamUtil.readStreamToString(resultStream)  
        // stream 처리 함수를 구현한 후 사용  
  
    } catch (e: Exception) {  
        // MalformedURLException, IOException, SocketTimeoutException 등 처리 필요  
        e.printStackTrace()  
    } finally {  
        if (resultStream != null) {  
            try { resultStream.close() }  
            catch (e: IOException) { Log.d(TAG, e.message!!) } }  
        if (conn != null) conn.disconnect()  
    }  
  
    return receivedContents  
}
```

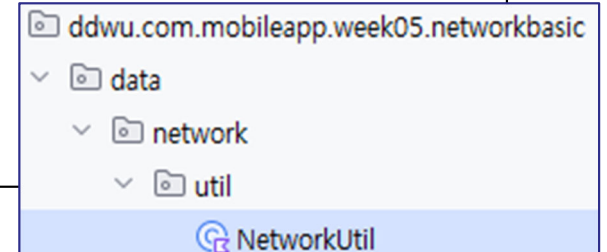
readStreamToString(...)을 사용하여 inputStream을 String으로 변환



GET 방식으로 이미지(Bitmap) 수신

```
fun downloadImage(address : String) : Bitmap? {  
    var receivedBitmap : Bitmap? = null  
    var resultStream : InputStream? = null  
    var conn: HttpURLConnection? = null  
  
    try {  
        conn = getConnection("GET", address, null)  
        resultStream = conn?.inputStream  
        receivedBitmap = streamUtil.readStreamToImage(resultStream)  
    } catch (e : Exception) {  
        e.printStackTrace()  
    } finally {  
        if (resultStream != null) {  
            try { resultStream.close() }  
            catch (e: IOException) { Log.d(TAG, e.message!!) } }  
        if (conn != null) conn.disconnect()  
    }  
  
    return receivedBitmap  
}
```

readStreamToImage(...)
를 사용하여 inputStream
을 Bitmap으로 변환



네트워크 데이터 전송 및 수신 4 – POST 요청

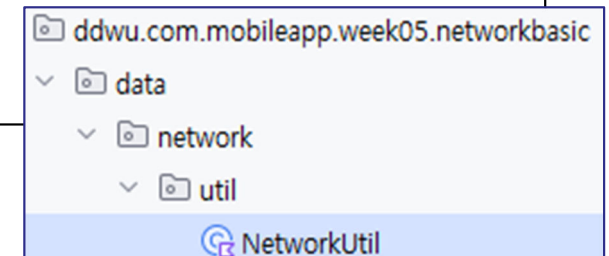
POST 방식으로 데이터 전송 및 결과 수신

```
fun sendPostData(requestMethod: String, address: String, data: String) : String? {
    var receivedContents : String? = null
    var resultStream : InputStream? = null
    var conn : HttpsURLConnection? = null

    try {

        conn = getConnection("POST", address, data)
        resultStream = conn?.inputStream // 응답 결과 스트림 확인
        receivedContents = streamUtil.readStreamToString (resultStream)
        // stream 처리 함수를 구현한 후 사용

    } catch (e: Exception) {
        // MalformedURLException, IOException, SocketTimeoutException 등 처리 필요
        e.printStackTrace()
    } finally {
        if (resultStream != null) {
            try { resultStream.close() }
            catch (e: IOException) { Log.d(TAG, e.message!!) } }
        if (conn != null) conn.disconnect()
    }
    return receivedContents
}
```



네트워크 연결 및 데이터 수신 3

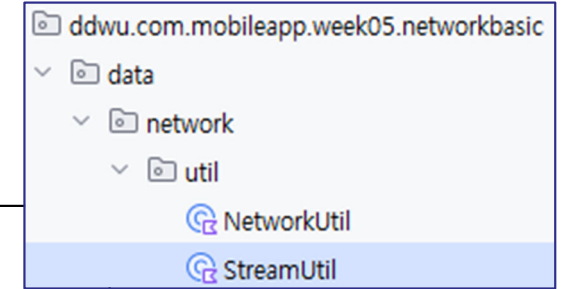
InputStream의 String 변환

```
private fun readStreamToString(iStream : InputStream?) : String {
    val resultBuilder = StringBuilder()

    val inputStreamReader = InputStreamReader(iStream)
    val bufferedReader = BufferedReader(inputStreamReader)

    var readLine : String? = bufferedReader.readLine()
    while (readLine != null) {
        resultBuilder.append(readLine + System.lineSeparator())
        readLine = bufferedReader.readLine()
    }

    bufferedReader.close()
    return resultBuilder.toString()
}
```



Stream →
StreamReader →
BufferedReader

"\n" ←

InputStream의 Bitmap 변환

```
private fun readStreamToImage(iStream: InputStream?) : Bitmap {
    val bitmap = BitmapFactory.decodeStream(iStream)
    return bitmap
}
```

Image 파일의 클 경우 오류가 발생할 수 있으므로 표시할 이미지의 크기 조절이 필요할 수 있음

출처: <https://developer.android.com/training/basics/network-ops/connecting.html>

HTTP 요청 설정

URLConnection 메소드	역할
<code>setConnectTimeout (timeout : Int) //connectTimeout</code>	1/1000 단위로 연결제한시간 설정, 0일 경우 무한 대기
<code>setReadTimeout (timeout : Int) // readTimeout</code>	1/1000 단위로 읽기 제한 시간 설정, 0일 경우 무한 대기
<code>setUseCaches (newValue : Boolean) // useCashes</code>	캐시 사용 여부를 지정한다.
<code>setDoInput (newValue : Boolean) // doInput</code>	입력을 받을 것인지를 지정한다.
<code>setDoOutput (newValue : Boolean) // doOutput</code>	출력을 할 것인지를 지정한다.
<code>setRequestProperty (field : String, newValue : String)</code>	요청 헤더에 값을 설정한다.
<code>addRequestProperty (field : String, newValue : String)</code>	요청 헤더에 값을 추가한다. 속성의 이름이 같더라도 덮어 쓰지는 않는다.

HTTP 요청 처리

URLConnection or HttpsURLConnection	역할
<code>setRequestMethod (method : String) // requestMethod</code>	GET 또는 POST 설정, 디폴트는 GET
<code>connect ()</code>	통신 연결 (네트워크 트래픽 발생)
<code>getResponseCode() : Int // responseCode</code>	서버에 요청을 전달하고 응답 결과를 받음 - 정상적으로 요청 전달: HTTP_OK (200) - URL을 발견할 수 없음: HTTP_NOT_FOUND(404) - 인증 실패: HTTP_UNAUTHORIZED (401) - 서비스 사용 불가: HTTP_UNAVAILABLE (503)
<code>getInputStream() // inputStream</code>	서버에서 전달받은 데이터를 InputStream 타입으로 반환
<code>disconnect()</code>	서버와의 연결 종료

권한 오류 처리

❏ Main Thread 에서의 네트워크 사용 금지

- ◆ 3.0 버전부터 사용자 응답성 문제로 인해 Main Thread (UI Thread)에서 네트워크 실행 금지 → 응답에 시간이 걸릴 경우 ANR(Application Not Responding) 발생 (강제종료)

❏ 해결 방법 (임시 및 테스트용)

- ◆ onCreate() 실행 시 아래 코드 실행

```
val pol = StrictMode.ThreadPolicy.Builder().permitNetwork().build()
StrictMode.setThreadPolicy(pol)
```

- ◆ 테스트나 임시로 네트워크를 확인해볼 때만 사용할 것
- ◆ Thread 또는 AsyncTask 등을 사용하여 별도의 스레드에서 네트워크 기능을 구현하는 방법으로 교체 필요

❏ 비동기 처리 필수! → Thread or Coroutine

HTTP 통신 유형 1 – GET

☞ URL 구성 시 매개변수(쿼리)를 URL에 추가

프로토콜://호스트주소[:포트번호]/[경로]/[파일]?{쿼리}

<http://www.kobis.or.kr/kobisopenapi/webservice/rest/boxoffice/searchDailyBoxOfficeList.xml?key=430156241533f1d058c603178cc3ca0e&targetDt=20120101>

☞ 쿼리 구성

- ◆ ?param_name=param_value&...
- ◆ HashMap<String, String>으로 구성하기가 용이
- ◆ strings.xml 에 기록할 경우 URL에 나타난 & → & 변환하여 입력

☞ REST API 의 경우

<http://cs.dongduk.ac.kr/bbs/board5>

실습 1

미완성 부분을 완성해볼 것

◆ 이미지 다운로드 부분을 완성

- [DOWNLOAD IMAGE] 버튼을 누르면 이미지를 다운로드하여 ImageView (ivImage)에 표시
- 이미지 다운로드 버튼 클릭 시 strings.xml 파일의 주소에 기록되어 있는 `R.string.image_url` 주소의 이미지를 다운로드

```
val url = resources.getString( R.string.STRING_ID )
```

◆ POST 요청 부분을 완성

- etData 에 값 입력 후 [SEND DATA] 버튼 클릭 시 실행
- `R.string.server_url`
- 화면의 etData 에 입력한 값을 server 에 post 로 전송
- 전송 결과 확인

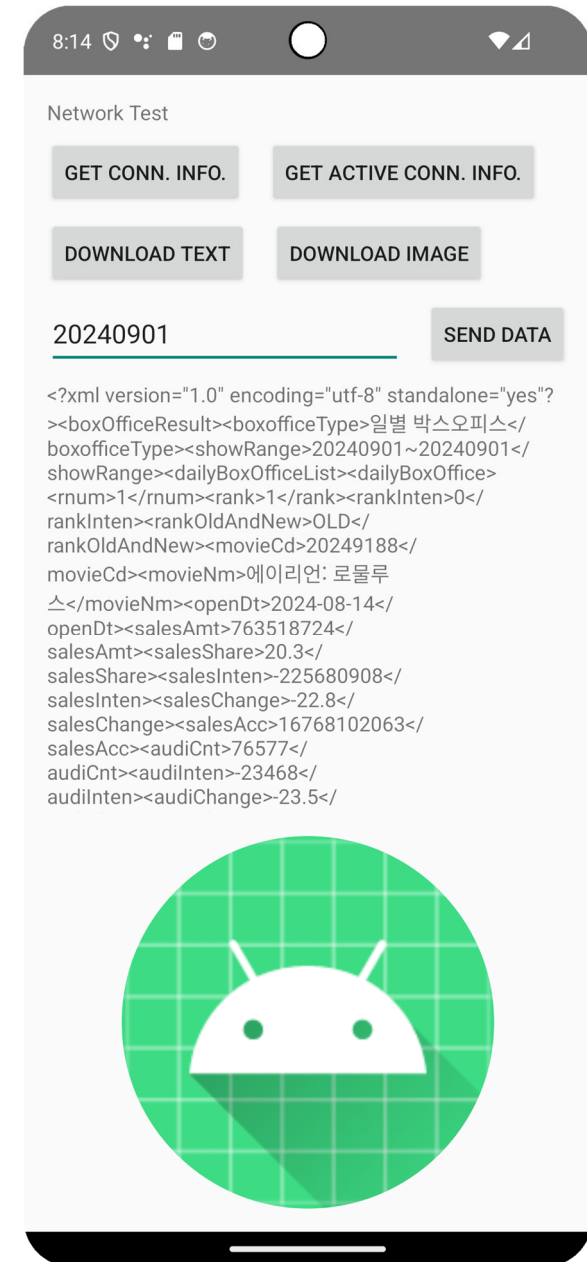
실습 2

영화진흥위원회 OpenAPI 를 이용해보기

- ◆ etData 에 날짜 입력 → 20XXXXX 형식
- ◆ 입력 후 [DOWNLOAD TEXT]
클릭 시 실행 GET 방식 요청
- ◆ strings.xml 의 movie_url 뒷 부분에
etData 에 입력한 날짜 결합
→ 연결주소에 전달

POST 방식 사용 시 여러 Params 사용 방법 고려해보기

- ◆ HashMap<key, value> 사용



참고

HTTP 요청을 확인해볼 수 있는 테스트 사이트

- ◆ 링크: <https://httpbin.org/>
- ◆ GET 과 POST 방식을 확인해볼 수 있음

영화진흥위원회 Open API 서비스

- ◆ 링크: <https://www.kobis.or.kr/kobisopenapi>
- ◆ GET 방식의 요청 결과 확인